Specs in the evening, reviewed branches in the morning.

A kit for spec-driven autonomous delivery with Claude Code. A person writes a spec. An agent plans, builds and reviews it in short loops that each start from a fresh context, while a queue runner carries several specs through the night and small scripts check every claim the agent makes.

NODE 20 · NO DEPENDENCIES WINDOWS · MACOS · LINUX MIT LICENCE

PERSON + AGENT	AGENT	AGENT, MANY TIMES	AGENT	PERSON
● drafted	● planned	● built	● reviewed	● shipped
`/design` interviews you, reads the code and writes the contract.	Gap analysis and 3–6 phases, each task naming its files and tests.	One task per iteration: failing tests first, then code, then a commit.	The whole diff read once, fixed in slices, findings sorted by scope.	You read it, sign what needs a name, and merge.
`lint-spec`	`verify-plan-completeness`	`verify-plan-ticks` · `acceptance-bindings` · `exit gate`	`gates` · `review agent` · `review cap`	`your branch protection`

Two ideas carry the design

1

An agent with an empty head cannot drift

Every iteration is a new process that reads its state from disk: the spec, the plan, a status file, git. Nothing important lives in a context window, so a crash, a reboot or a bad iteration costs one iteration, not the night.

2

An agent's claim is not evidence

Each claim that matters has a script that checks it against the files, the tests or git, and undoes the claim if it does not hold. The agent can say a phase is done; the phase is ticked only when the check agrees.

THE AGENT SAYS	CHECKED BY	IF IT DOES NOT HOLD
"The plan covers the spec."	`verify-plan-completeness`	Lists every requirement no task builds; the plan cannot be marked ready.
"This phase is done."	`verify-plan-ticks`	Un-ticks the phase and writes down why.
"This test proves the criterion."	`verify-acceptance-bindings`	Flags the row as missing, skipped, assertion-free or tested at the wrong layer.
"Those failures were already there."	`base-health --compare`	Measures the base branch in its own worktree; only a regression fails.
"The tests pass." (with no test command set)	the Exit Gate	Reported as "not configured", never as passing.

The loop technique itself, Ralph, is Geoffrey Huntley's: run a coding agent in a loop against a prompt and files on disk. This kit is one team's answer to what it takes to leave that running unattended on a real codebase.

What a night looks like

`Illustrative timeline · times and spec names are examples`

17:40 You tell Claude Code: "work autonomously through these three specs". The `autonomous` skill runs its pre-flight: clean index, no other loop, doctor, how far the branch has drifted from main.

17:44 It writes the queue and starts a detached runner that will outlive the session and keep the machine awake.

```
- [ ] specs/team-invitations.md
- [ ] specs/export-verbose-flag.md
- [ ] specs/audit-log-retention.md
```

17:52 Plan mode on the first spec: gap analysis, base-branch health, five phases. The completeness check passes. Ledger: `planned`.

18:00 Build iterations begin, three per invocation. Each writes failing tests named exactly as the spec names them, implements, runs only the checks that prove its slice, and commits.

21:15 The second spec hits a question only a person can answer. The loop records it and exits `blocked`; the runner parks it and moves on.

```
- [!] specs/export-verbose-flag.md — blocked in build: [product] should --verbose include timings?
```

23:23 The OS kills the loop under memory pressure. The runner waits for free memory and resumes from the ledger. Nothing is lost but one iteration.

02:40 First spec reaches its Exit Gate: full lint, typecheck, build and test, every acceptance row bound, one review by the review agent. Ledger: `built`. Review passes follow; a clean one sets `reviewed`.

08:30 You read the talk log. It leads with what needs you (one product question, one judgement call flagged `[REVIEW]`), then what finished. You answer, set the parked line back to `[ ]`, and open the draft PR for the reviewed spec.

What to expect

Dozens of iterations per spec

Not a handful. A small spec with a clean contract finishes quickly; one that crosses several layers or needs end-to-end tests takes much longer, and most of the extra is re-verifying, not writing code.

Review is the expensive mode

It runs on the largest model and reads the whole diff, and left alone it keeps finding things. The review cap and the scope audit exist to stop it running on.

Real specs run to a few hundred lines

That is what it takes for the loop to build without asking: the mandatory questions answered, acceptance tests on every phase, and a scope that says what is out. The repository includes one at realistic size.

Most overnight stops are the environment

An end-to-end stack that is down, a test service out of quota, a VPN that is not up, a waiver only a person can sign. The loop is only as autonomous as the environment its tests need.

What is automated, and what stays yours

THE KIT DOES

- Planning, building and reviewing, and the checks on all three
- Carrying a queue of specs from one to the next
- Recovering from crashes, reboots and usage limits
- Keeping the machine awake and killing orphaned processes
- Deciding what your written standards already decide, and logging each call
- Telemetry: time, cost and outcome per iteration

A PERSON DOES

- Decides what the feature is: every question tagged `[product]`
- Signs approvals and waivers; they carry a name
- Answers parked specs and sets them back to `[ ]`
- Runs the scope audit when review hits its cap
- Reads the talk log for judgement calls made in their absence
- Merges and ships

The plan is deliberately not a manual gate. The checks decide whether a plan covers its spec; you may read and edit it, but nothing waits for you.

Scope: what it builds, and what it files instead

A spec's problem statement, scope and success criteria are a boundary the loops never widen. Every finding from build or review is sorted before anyone acts on it.

THE FINDING IS	WHAT HAPPENS	WHERE YOU SEE IT
Inside the spec's scope	Fixed now, properly, with a test.	The fix commit
Outside, but ship-blocking: this spec's deliverable would be unsafe or wrong without it	Fixed now, and flagged with the reason it could not wait.	`[REVIEW: SCOPE]` in the commit and the decision report
Outside and pre-existing: true before this spec touched anything	**Not built.** Filed as a candidate spec with the residual risk; a linked Jira issue when Jira is on.	DEFERRED in the decision report

Only a person widens a spec. If review keeps finding work, it stops after five passes and asks one batched question: accept the risk, file follow-up specs, or widen the spec. The queue parks the spec after eight review passes regardless.

Decisions it makes, and how you review them

WHO	DECIDES	RECORDED AS
You	What the feature is. Approvals and waivers. Merging, releasing, infrastructure, anything outward-facing.	Your answer in the spec's Open Questions, dated
The supervising skill	Only what your written standards or recorded decisions already decide.	`[REVIEW]` entries naming the rule used and how to undo it
The loops	How to build: where code goes, which pattern, how to split a task. Fixes to their own tooling.	Numbered plan decisions (`D7`) with the rule behind each; `Tooling:` commits

The records are committed with the work: a spec's Open Questions grow into a dated decision log, the plan carries its decisions, commit messages carry the rest. When you come back, one command lists everything, with what needs you first:

```
node scripts/ralph-decisions.mjs
```

```
NEEDS YOU (2)
  parked        specs/export-flag.md — blocked in build: [product]
include timings?
  question      [product] Q6 — restrict invitations to verified
domains?

DECIDED FOR YOU (2)
  agent call    [product] [REVIEW] Q4 — 404 for non-members, from
standards/security
  plan decision D11 - expiry sweep runs in the existing job scheduler

DEFERRED (1)
  candidate     CANDIDATE-SPEC: audit export has no paging
```

Example output

Mark a decision `[REVIEWED]` when you accept it and it drops out of the report. To overturn one, change it in the spec or plan and re-queue; the next iteration reads the new answer. A pattern of wrong calls becomes a rule in `standards/`, and every later spec follows it.

Lanes: specs in parallel

One worktree runs one loop at a time. A lane is a sibling worktree on its own branch with its own queue and runner, so two or three independent specs build at once.

When to open one

Independent specs that touch different files; a machine with room for another agent and test run; and an environment the lane can have to itself: its own ports, databases and test accounts. Two or three lanes at most, the main worktree included, and never a lane of a lane.

How to split

By the files specs touch: backend in one lane, frontend in another. Specs that depend on each other stay in the same lane, in order. Specs that need the same scarce test environment share a lane too.

When to merge back

After each spec reaches `reviewed`, one lane at a time, with a person's approval. Stop both runners, merge, keep the main plan, merge the ledger row by row, verify, then bring every other lane up to date straight away.

What to expect

With a good split, merges conflict only on the plan and ledger files, and a checklist script walks through them. A lane left unmerged for a week drifts far enough that the merge becomes its own project.

Jira, if you use it

Off by default; nothing mentions a tracker. Turned on, each spec is bound to an issue and the loops keep it current without per-iteration noise.

WHEN	IN JIRA
The spec is written	Issue created or bound; moves to ready for development
Building starts	Moves to in progress
The exit gate passes	Moves to review required, with each gate's result
Each review pass	A comment: fixes applied, items left for a person, findings deferred

WHEN	IN JIRA
An out-of-scope finding worth tracking	A new linked issue

The loops never close, delete, reassign or reprioritise issues. If Jira is unreachable they keep working, and commits are refused until the missed update is retried, so no ticket is silently left behind.

Adopting it

Allow an hour for the first spec. Start with a small one, on a branch, in a sandbox.

1 Copy the kit into your repository

`ralph/`, `scripts/`, `.githooks/`, `.claude/`, `standards/`, `templates/`, `docs/`. Merge into folders you already have; the getting-started guide lists what to watch for.

2 Configure your commands and check the setup

```
cp templates/config.example.json ralph.config.json
node scripts/install-hooks.mjs
node scripts/ralph-doctor.mjs
```

Set your test, lint, typecheck and build commands. A command left empty is reported "not configured", never green. The hook installer refuses to replace hooks you already have.

3 Write five rules in `standards/`

The five things you would otherwise repeat in every code review. The spec author, the builder and the review agent all read them; every rule written down is a finding the loop stops making.

4 Write a spec

```
/design "add a --verbose flag to the export command"
```

An interview, codebase research, seven mandatory questions (events, permissions, external exposure, flags, agent behaviour, UI states, migration), then a spec with acceptance tests per phase.

5 Run the loops by hand once

```
./ralph/loop.sh plan   1  specs/my-spec.md
./ralph/loop.sh build  20 specs/my-spec.md
./ralph/loop.sh review 5  specs/my-spec.md
```

6 Then queue a few and walk away

In Claude Code: "work autonomously through specs/a.md and specs/b.md". Keep the queue to about five specs; a long queue starts more work than it finishes.

> **Before the first overnight run**
>
> Unattended runs skip permission prompts: the agent can run any command with your user's rights. The prompts forbid force pushes, infrastructure changes and signing approvals, but those are instructions, not a sandbox.
>
> - Run it in a container, VM or throwaway worktree with no production credentials or deploy tooling.
> - Work on a branch; keep the pull request a draft so each push does not run your full CI.
> - Tag a checkpoint first, so undoing the whole run is one `git reset`.
> - Check your `CLAUDE.md` for rules like "ask before committing" that contradict an unattended loop.

Each rule is there because something went wrong

RULE IN THE KIT	WHAT HAPPENED WITHOUT IT
The queue runner is detached and keeps the machine awake	A process reaped under memory pressure at 23:23 lost eight hours of overnight work.
Review is capped, then audited for scope	One spec went through 34 review passes over eight days, each fixing what the last one introduced.
Verify a park reason against primary evidence	Loops diagnosed a wedged dependency as a warm-up race, then as throttling, for eight hours.
Keep the queue to about five specs	Seventeen large specs queued at once produced hundreds of commits and only three reviewed specs.

RULE IN THE KIT	WHAT HAPPENED WITHOUT IT
Every background process starts through one process owner	More than 60 orphaned headless browsers on one machine in a day.
A missing check is "not configured", never "pass"	A hook tested for exit code 1 only, so a command that was not found passed.
Draft pull requests while loops push	Every slice pushed to a ready PR ran the full CI suite, almost all of it against work in progress.

In the repository

github.com/centerMikeHicks/ralph-autonomous

`README.md`
The overview, with diagrams

`GETTING_STARTED.md`
Empty checkout to shipped spec

`docs/DECISIONS.md`
Decision rights, scope rules, review

`docs/LANES.md`
Parallel specs, split and merge

`docs/ARCHITECTURE.md`
Each piece and the failure it guards

`examples/specs/`
A spec at realistic size

Requires Claude Code, Node.js 20+ and git. Other agent CLIs can be configured but lose telemetry and usage-limit recovery.

The Ralph loop technique is Geoffrey Huntley's (ghuntley.com/ralph). The kit is by Mike Hicks, MIT licensed.